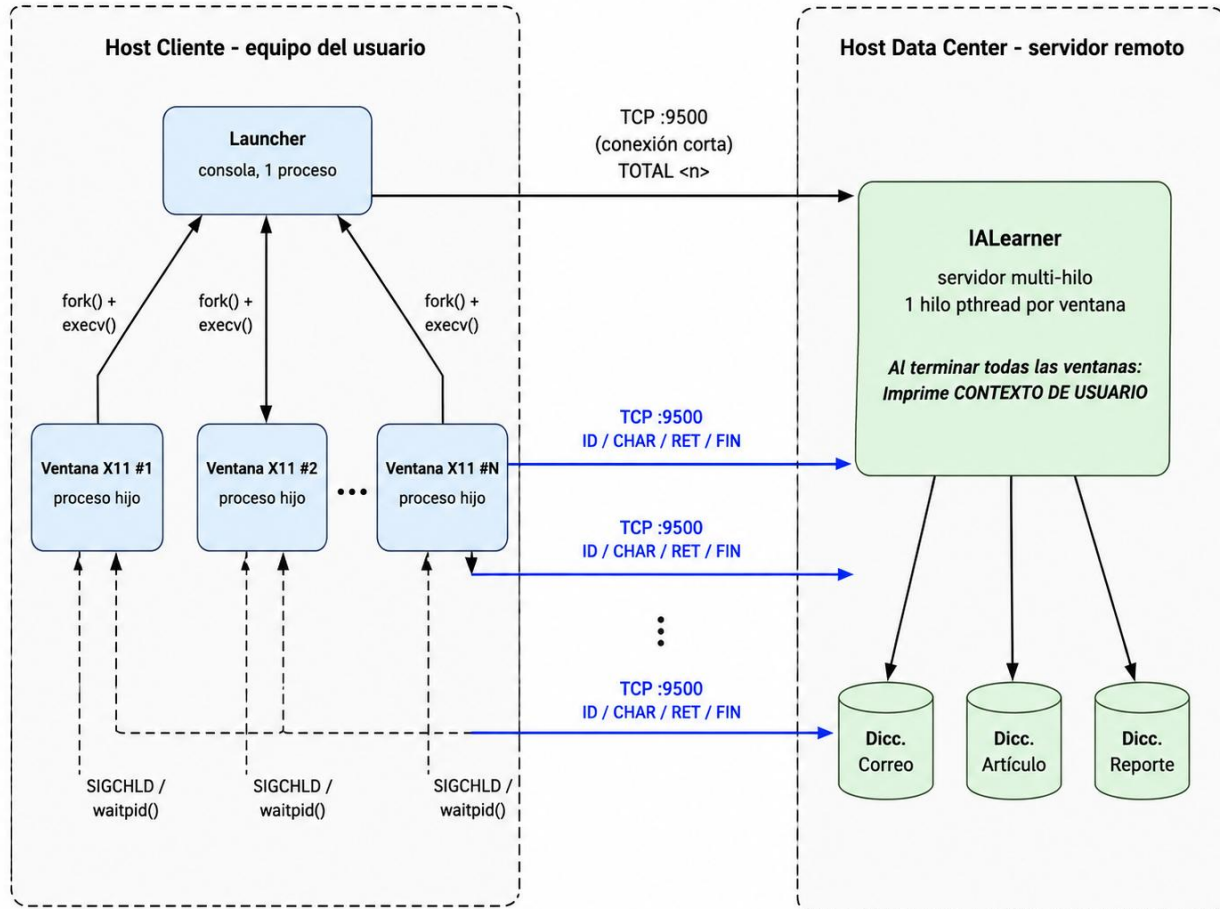


Documento de Diseño — Agentic-OS

Autor: LUIS ERNESTO ROCA MACIAS

Diagrama de Despliegue



1. Arquitectura General

El sistema Agentic-OS emplea una arquitectura distribuida separada en dos entornos físicos o lógicos: el Host Cliente (equipo del usuario) y el Host Data Center (servidor remoto).

En el Host Cliente, el proceso Launcher interactúa con el usuario y orquesta la creación de N procesos hijos (Ventana X11).

En el Data Center, el proceso IA Learner actúa como un servidor TCP multi-hilo que recibe la información en tiempo real de cada ventana de forma asíncrona.

Esta separación garantiza que el procesamiento pesado de clasificación textual no afecte el rendimiento de la interfaz gráfica del usuario.

2. Mecanismos de Multiprogramación (Eficiencia)

El diseño garantiza un alto rendimiento mediante el uso de multiprocesamiento local y multihilo remoto:

Local (Launcher y Ventanas): La generación de ventanas se realiza mediante llamadas a `fork()` + `execv()`, permitiendo que cada interfaz gráfica corra en su propio espacio de memoria. Para la gestión del ciclo de vida, el Launcher implementa un manejador de señales `SIGCHLD` acoplado a un bucle `waitpid(-1, &estado, WNOHANG)`. Esto permite recolectar múltiples procesos hijos que terminan simultáneamente sin bloquear la ejecución del menú interactivo y previniendo la aparición de procesos zombie.

Remoto (Concurrencia en IALearner): El servidor implementa el modelo de un hilo por conexión mediante `pthreads` en estado `detached`. Cada hilo posee su propia estructura de datos (`RegistroDocumento`) protegida por un cerrojo de exclusión mutua (`mutex`) individual. Esto asegura que las letras enviadas simultáneamente desde distintas ventanas se procesen de forma segura y sin entrelazarse, garantizando la consistencia de los datos. Un `mutex` y una variable de condición globales coordinan de forma eficiente el momento exacto en que se dispara el cálculo final del usuario.

3. Mecanismos de IPC (Comunicación entre Procesos)

Dado que los procesos residen en hosts distintos, la comunicación por tuberías locales es inviable; por ello, se implementó un mecanismo IPC basado en sockets TCP.

El Launcher establece una conexión corta por el puerto 9500 exclusivamente para transmitir el comando `TOTAL <n>`, informando al servidor la cantidad exacta de documentos que debe procesar.

Paralelamente, cada proceso Ventana X11 abre una conexión persistente hacia el IALearner por el mismo puerto. Se diseñó un protocolo de texto ligero basado en prefijos de una línea terminados en `\n` (`ID`, `CHAR`, `RET`, `FIN`), lo que minimiza la sobrecarga de red y permite una transmisión carácter por carácter altamente eficiente.

4. Algoritmos de Clasificación (Bag of Words)

El procesamiento del texto entrante utiliza la técnica Bag of Words. Las secuencias de texto recibidas hasta encontrar un salto de línea se transforman en vectores de frecuencias que se evalúan contra tres diccionarios en memoria (`Correo`, `Artículo`, `Reporte`).

Regla de pertenencia: Un documento se clasifica dentro de una categoría si presenta coincidencias con al menos tres palabras distintas del diccionario correspondiente. En caso de calificar para múltiples clases, el algoritmo de desempate selecciona aquella con la mayor suma total de frecuencias.

Nota de Diseño (Inferencia de Usuario): El requerimiento establecía el uso de "proporciones" para inferir el perfil final del usuario (`Administrativo`, `Técnico`, `Profesor`, `Estudiante`). Al analizar la matriz de referencia, se determinó que los cuatro perfiles poseen combinaciones binarias de presencia/ausencias estrictamente mutuamente excluyentes. Por eficiencia y precisión, el algoritmo interpreta las variables como una presencia binaria (al menos un documento clasificado en dicha clase), eliminando cualquier posible ambigüedad matemática.

5. Programación Defensiva y Liberación de Recursos

La robustez del código se asegura mediante prácticas defensivas en todos los componentes:

Ambos procesos principales validan estrictamente sus parámetros de entrada y no sufren caídas abruptas ante configuraciones inválidas.

El cierre de recursos es explícito: todos los descriptores de sockets y displays de X11 se cierran correctamente, y los mutex se destruyen al finalizar la ejecución del servidor.

El sistema de apagado del Launcher envía la señal SIGTERM para un cierre coordinado, escalando a SIGKILL como medida de respaldo para procesos bloqueados, liberando completamente la memoria y la CPU.